

geracao

May 19, 2026

Geração de Números Aleatórios em Python

João Gabriel Miguêz da Silva - 2023008890 Felipe Bastos Vargas - 2023008881

1 1.Introdução

Este trabalho tem como objetivo testar de diferentes modelos de distribuição para dados gerados aleatoriamente

Dentre as distribuições discretas serão aplicadas:

1. **Uniforme Discreta**
2. **Bernoulli**
3. **Binomial**
4. **Poisson**

Já as distribuições contínuas serão:

1. **Uniforme contínua**
2. **Normal**
3. **Exponencial**
4. **Gama**
5. **Beta**
6. **t-Student**

2 Metodologia

Inicialmente geramos aleatoriamente conjunto de dados usando a semente aleatória 42 para cada distribuição.

```
[2]: import numpy as np
from scipy . stats import t
import pandas as pd
import math

np.random.seed (42)

# Distribuicoes discretas
ddsUniformeDiscreta = np.random.randint(0,10,size=1000) #uniforme
↳discreta
```

```

ddsBernoulli = np.random.binomial(n =1,p =0.5,size =1000)           #Bernoulli
ddsBinomial = np.random.binomial(n =10,p =0.5,size =1000)          #Binomial
ddsPoisson = np.random.poisson(lam=5,size =1000)                    #Poisson

# Distribuicoes continuas
ddsUniformeContinua = np.random.uniform(0,1,size =1000)            #uniforme
↳ continua
ddsNormal = np.random.normal(0,1,size =1000)                       #normal
ddsExponencial = np.random.exponential(scale =2,size =1000)         #exponencial
ddsGamma = np.random.gamma(shape =2,scale =2,size =1000)           #gama
ddsBeta = np.random.beta( a =2,b =5,size =1000)                    #beta
ddsTStudent= t.rvs(df =5,size =1000)                                #
↳ t-Student

```

Em seguida analisamos as frequências observadas dos dados de cada conjunto. Para o conjunto de dados discretos, criamos um dicionário para que fosse possível contabilizar a frequência através de um loop.

2.1 Frequencias observadas dados discretos

```

[3]: #Frequencias dos dados Discretos
frqObservadaUniformeDiscreta = {}
frqObservadaBernoulli = {}
frqObservadaBinomial = {}
frqObservadaPoisson = {}

conjuntoDeDadosDiscretos = [(ddsUniformeDiscreta, frqObservadaUniformeDiscreta),
                             (ddsBernoulli, frqObservadaBernoulli),
                             (ddsBinomial, frqObservadaBinomial),
                             (ddsPoisson, frqObservadaPoisson)]

for dados, frq in conjuntoDeDadosDiscretos:
    for valor in dados:
        if valor in frq:
            frq[valor] += 1
        else:
            frq[valor] = 1

```

2.2 Frequencias observadas dados continuos

```

[4]: #Continuas
frqObservadaUniformeContinua = {}
frqObservadaNormal = {}
frqObservadaExponencial = {}
frqObservadaGamma = {}

```

```

frqObservadaBeta = {}
frqObservadaTStudent = {}

conjuntoDeDadosContinuos = [(ddsUniformeContinua, frqObservadaUniformeContinua),
                             (ddsNormal, frqObservadaNormal),
                             (ddsExponencial, frqObservadaExponencial),
                             (ddsGamma, frqObservadaGamma),
                             (ddsBeta, frqObservadaBeta),
                             (ddsTStudent, frqObservadaTStudent)]

```

2.3 Cálculo das frequências esperadas nas distribuições

2.3.1 Distribuição Uniforme Discreta

```

[5]: n = 10          # número de valores possíveis
     a = 0          # valor mínimo
     b = a + n - 1  # valor máximo possível (0 + 10 - 1 = 9)

def FMP_DiscretaUniforme(n, a, b):

    valores_x = [i for i in range(a, b + 1)]
    probabilidade = 1 / n
    probabilidades = [probabilidade for _ in range(n)]
    fmp_resultado = dict(zip(valores_x, probabilidades))

    return fmp_resultado

def calcular_estatisticas_uniforme(a, b):
    esperanca = (a + b) / 2
    variancia = ((b - a + 1)**2 - 1) / 12
    return esperanca, variancia

fmpDiscretaUniforme = FMP_DiscretaUniforme(n, a, b)
mediaTeoricaDiscUni, varTeoricaDiscUni = calcular_estatisticas_uniforme(a, b)

```

2.3.2 Distribuição de Bernoulli

```

[6]: pSucesso = 0.5
     pFracasso = 1 - pSucesso

def FMP_Bernoulli(p):

    valores_x = [0, 1]
    probabilidades = [1 - p, p]
    fmp_resultado = dict(zip(valores_x, probabilidades))
    return fmp_resultado

```

```
def calcular_estatisticas_bernoulli(p):
    esperanca = p
    variancia = p * (1 - p)
    return esperanca, variancia

fmpBernoulli = FMP_Bernoulli(pSucesso)
mediaTeoricaBernoulli, varTeoricaBernoulli = _
↪calcular_estatisticas_bernoulli(pSucesso)
```

2.3.3 Distribuição Binomial

```
[7]: n = 10    # Número de tentativas independentes
     p = 0.5    # Probabilidade de sucesso em cada tentativa

def FMP_Binomial(n, p):
    fmp_resultado = {}
    for k in range(n + 1):
        coeficiente = math.comb(n, k) #  $\binom{n}{k}$ 
        ↪coeficiente binomial:  $n! / (k! * (n - k)!)$ 
        probabilidade = coeficiente * (p**k) * ((1 - p)**(n - k)) #  $P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$ 
        ↪Coeficiente *  $p^k * (1-p)^{n-k}$ 
        # Guardar no dicionário
        fmp_resultado[k] = probabilidade
    return fmp_resultado

def calcular_estatisticas_binomial(n, p):
    esperanca = n * p
    variancia = n * p * (1 - p)
    return esperanca, variancia

fmpBinomial = FMP_Binomial(n, p)
mediaTeoricaBinomial, varTeoricaBinomial = calcular_estatisticas_binomial(n, p)
```

2.3.4 Distribuição Poisson

```
[8]: lam = 5    # Lambda

def FMP_Poisson(lam, limite_max=15):
    fmp_resultado = {}
    for k in range(limite_max + 1):

        fatorial_k = math.factorial(k)
        probabilidade = (math.exp(-lam) * (lam**k)) / fatorial_k #  $P(X = k) = \frac{e^{-\lambda} \lambda^k}{k!}$ 
        ↪ $(e^{-\lambda} * \lambda^k) / k!$ 
```

```

        # Guardar no dicionário
        fmp_resultado[k] = probabilidade
    return fmp_resultado

def calcular_estatisticas_poisson(lam):
    return lam, lam

fpmPoisson = FMP_Poisson(lam)
mediaTeoriaPoisson, varTeoricaPoisson = calcular_estatisticas_poisson(lam)

```

3 Analise dos Resultados

```

[11]: tabelas_frequencia = [
    ("Uniforme Discreta", frqObservadaUniformeDiscreta, fmpDiscretaUniforme),
    ("Bernoulli", frqObservadaBernoulli, fmpBernoulli),
    ("Binomial", frqObservadaBinomial, fmpBinomial),
    ("Poisson", frqObservadaPoisson, fpmPoisson)
]

for nome, frq_dict, fmp_dict in tabelas_frequencia:
    print(f"\n" + "="*70)
    print(f" TABELA COMPARATIVA: {nome.upper()} ")
    print("="*70)

    df = pd.DataFrame(list(frq_dict.items()), columns=['Valor', 'Freq. Obs. Absoluta (f_o)'])
    df = df.sort_values(by='Valor').reset_index(drop=True)

    total_observacoes = df['Freq. Obs. Absoluta (f_o)'].sum()

    df['Prob. Teórica P(X=x)'] = df['Valor'].map(fmp_dict)
    df['Prob. Teórica P(X=x)'] = df['Prob. Teórica P(X=x)'].fillna(0)
    df['Freq. Esp. Absoluta (f_e)'] = df['Prob. Teórica P(X=x)'] * total_observacoes
    df['Freq. Obs. Relativa (f_r)'] = df['Freq. Obs. Absoluta (f_o)'] / total_observacoes

    df_formatado = df.copy()

    df_formatado['Freq. Esp. Absoluta (f_e)'] = df_formatado['Freq. Esp. Absoluta (f_e)'].map("{:.1f}".format)
    df_formatado['Prob. Teórica P(X=x)'] = df_formatado['Prob. Teórica P(X=x)'].map("{:.2%}".format)

```

```

df_formatado['Freq. Obs. Relativa (f_r)'] = df_formatado['Freq. Obs. Absoluta (f_o)'] / df_formatado['Prob. Teórica P(X=x)']
df_formatado['Freq. Obs. Relativa (f_r)'].map("{:.2%}".format)

colunas_ordenadas = [
    'Valor',
    'Freq. Obs. Absoluta (f_o)', 'Freq. Esp. Absoluta (f_e)',
    'Freq. Obs. Relativa (f_r)', 'Prob. Teórica P(X=x)'
]
print(df_formatado[colunas_ordenadas].to_string(index=False))

```

=====

TABELA COMPARATIVA: UNIFORME DISCRETA

=====

Valor (f_r)	Freq. Obs. Absoluta (f_o)	Freq. Esp. Absoluta (f_e)	Freq. Obs. Relativa (f_r) Prob. Teórica P(X=x)
0	118	100.0	
11.80%	10.00%		
1	83	100.0	
8.30%	10.00%		
2	110	100.0	
11.00%	10.00%		
3	94	100.0	
9.40%	10.00%		
4	107	100.0	
10.70%	10.00%		
5	96	100.0	
9.60%	10.00%		
6	94	100.0	
9.40%	10.00%		
7	100	100.0	
10.00%	10.00%		
8	91	100.0	
9.10%	10.00%		
9	107	100.0	
10.70%	10.00%		

=====

TABELA COMPARATIVA: BERNOULLI

=====

Valor (f_r)	Freq. Obs. Absoluta (f_o)	Freq. Esp. Absoluta (f_e)	Freq. Obs. Relativa (f_r) Prob. Teórica P(X=x)
0	489	500.0	
48.90%	50.00%		
1	511	500.0	
51.10%	50.00%		

=====

TABELA COMPARATIVA: BINOMIAL

Valor (f_r)	Freq. Obs. Prob. Teórica	Absoluta (f_o)	Freq. Esp. Absoluta (f_e)	Freq. Obs. Relativa
0		1	1.0	
0.10%				
1		12	9.8	
1.20%				
2		50	43.9	
5.00%				
3		125	117.2	
12.50%				
4		175	205.1	
17.50%				
5		246	246.1	
24.60%				
6		218	205.1	
21.80%				
7		115	117.2	
11.50%				
8		44	43.9	
4.40%				
9		12	9.8	
1.20%				
10		2	1.0	
0.20%				

TABELA COMPARATIVA: POISSON

Valor (f_r)	Freq. Obs. Prob. Teórica	Absoluta (f_o)	Freq. Esp. Absoluta (f_e)	Freq. Obs. Relativa
0		7	6.7	
0.70%				
1		31	33.7	
3.10%				
2		88	84.2	
8.80%				
3		147	140.4	
14.70%				
4		177	175.5	
17.70%				
5		173	175.5	
17.30%				
6		151	146.2	
15.10%				
7		84	104.4	
8.40%				

8		67	65.3
6.70%	6.53%		
9		45	36.3
4.50%	3.63%		
10		15	18.1
1.50%	1.81%		
11		8	8.2
0.80%	0.82%		
12		5	3.4
0.50%	0.34%		
13		1	1.3
0.10%	0.13%		
15		1	0.2
0.10%	0.02%		